

Non-Equi Joins

Non-equi joins use operators like <, >, BETWEEN for ranges (e.g., salary bands, thresholds). Use these tables :

non_equi_sales.csv (6 orders):

order_id	customer_id	amount	order_date
101	C1	1500	2025-01-15
102	C2	800	2025-02-10
103	C1	2500	2025-03-20
104	C3	1200	2025-01-25
105	C2	3000	2025-04-05
106	C4	900	2025-02-28

non_equi_bands.csv (4 salary bands):

band_id	min_salary	max_salary	band_name
1	0	1000	Low
2	1001	3000	Medium
3	3001	5000	High
4	5001	99999	Executive

non_equi_emp.csv (5 employees):

emp_id	name	salary
1	Alice	2500
2	Bob	4500
3	Charlie	7500

4	Diana	1200
5	Eve	6000

Exercise 1: Find employees whose salary falls in each band (join emp on bands where salary >= min_salary AND salary <= max_salary). Show emp name, salary, band_name.

Exercise 2: Identify high-value orders (>2000) and classify handling employees by salary band (join sales.amount >2000 with emp.salary and bands ranges). Output order_id, amount, emp name, band_name.

Exercise 3: Find orders where amount is between employee salary and salary*1.2 (non-equi on amount). Show order_id, amount, emp_id, name.

Exercise 4: Create a commission table (emp_id, min_amount, commission_rate) and join with sales where amount >= min_amount to compute commissions.

Exercise 5: Advanced: Rank orders by amount within each salary band (join sales, emp, bands with ranges, then use ROW_NUMBER() OVER (PARTITION BY band_name ORDER BY amount DESC)).

Left Outer Join

Left outer join keeps all left table rows, NULLs on right for no match. Use :

customers.csv (6 customers):

cust_id	cust_name	city
1	Raj	Delhi
2	Priya	Mumbai
3	Amit	Delhi
4	Sneha	Bangalore
5	Vikram	Chennai
6	Neha	Mumbai

orders.csv (5 orders):

order_id	cust_id	product	quantity
101	1	Laptop	1
102	3	Phone	2
103	2	Tablet	1
104	5	Watch	3
105	7	TV	1

Exercise 1: List all customers with their orders (LEFT JOIN customers ON orders.cust_id). Show cust_name, order_id (NULL if no order).

Exercise 2: Find customers without orders (WHERE orders.order_id IS NULL). Count them.

Exercise 3: Compute total quantity per customer (GROUP BY cust_id), showing 0 for customers without orders.

Exercise 4: Add city-wise customer count with orders summary (GROUP BY city).

Exercise 5: Find inactive customers (no orders in last 6 months, assuming order_date column added) using LEFT JOIN and date functions.

Right Outer Join

Right outer join keeps all right table rows, NULLs on left for no match. Use same tables .

Exercise 1: List all orders with customer details (RIGHT JOIN orders ON customers.cust_id). Show order_id, cust_name (NULL for cust_id=7).

Exercise 2: Identify orphan orders (customers.cust_name IS NULL). Explain business implication.

Exercise 3: Total quantity per product across all orders, including customer city if available.

Exercise 4: Find products ordered by customers from each city (GROUP BY city), NULL for no customers.

Exercise 5: Generate order aging report (days since order) with customer info, handling orphans.

Full Outer Join

Full outer join keeps all rows from both tables, NULLs where no match. Use .

Exercise 1: Complete customer-order view (FULL OUTER JOIN ON cust_id). Show all customers and orders.

Exercise 2: Count total unique cust_id across both tables (including orphans on both sides).

Exercise 3: City-wise summary: customers and total orders (GROUP BY city), handling NULLs.

Exercise 4: Find discrepancies: customers without orders OR orders without customers.

Exercise 5: Reconciliation report: Match customers to orders, flag mismatches (cust_id exists in one but not other), compute potential revenue (quantity * avg price=2000).